

Session 22- Optional Task

- **Name of Student :** Shruti Dnyaneshwar Darwatkar
- **Domain Name:** AIML Development
- **College Name:** Zeal Polytechnic , Narhe
- **Branch Name:** AIML
- **Github url:** https://github.com/shrutidarwatkar-sketch/Task_ITR.git

CODES:

Q1. Data Loading & Preprocessing

▶ #Q1

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# Load Dataset
from google.colab import files
uploaded = files.upload()

# Display first 5 rows
df = pd.read_csv("weatherAUS.csv")
print(df.head())

# Check missing values
print(df.isnull().sum())

# Fill numerical missing values with median
num_cols = df.select_dtypes(include=['float64', 'int64']).columns

for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

# Fill categorical missing values with mode
cat_cols = df.select_dtypes(include=['object']).columns

for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Drop Date column
df.drop("Date", axis=1, inplace=True)
```

```
# Drop Date column
df.drop("Date", axis=1, inplace=True)

# Label Encoding
encoder = LabelEncoder()

for col in df.select_dtypes(include='object').columns:
    df[col] = encoder.fit_transform(df[col])

print(df.head())
```

```
... Choose files weatherAUS.csv
weatherAUS.csv(text/csv) - 14094055 bytes, last modified: 21/07/2026 - 100% done
Saving weatherAUS.csv to weatherAUS (1).csv
```

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	\
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	

	WindGustDir	WindGustSpeed	WindDir9am	...	Humidity9am	Humidity3pm	\
0	W	44.0	W	...	71.0	22.0	
1	NNW	44.0	NNW	...	44.0	25.0	
2	WSW	46.0	W	...	38.0	30.0	
3	NE	24.0	SE	...	45.0	16.0	
4	W	41.0	ENE	...	82.0	33.0	

	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	Temp9am	Temp3pm	RainToday	\
0	1007.7	1007.1	8.0	NaN	16.9	21.8	No	
1	1010.6	1007.8	NaN	NaN	17.2	24.3	No	
2	1007.6	1008.7	NaN	2.0	21.0	23.2	No	
3	1017.6	1012.8	NaN	NaN	18.1	26.5	No	
4	1010.8	1006.0	7.0	8.0	17.8	29.7	No	

	RainTomorrow
0	No
1	No
2	No
3	No
4	No

```
... [5 rows x 23 columns]
Date      0
Location  0
MinTemp   1485
MaxTemp   1261
Rainfall  3261
Evaporation 62790
Sunshine  69835
WindGustDir 10326
WindGustSpeed 10263
WindDir9am  10566
WindDir3pm  4228
WindSpeed9am 1767
WindSpeed3pm 3062
Humidity9am  2654
Humidity3pm  4507
Pressure9am  15065
Pressure3pm  15028
Cloud9am     55888
Cloud3pm     59358
Temp9am      1767
Temp3pm      3609
RainToday    3261
RainTomorrow 3267
dtype: int64
```

```
df[col].fillna(df[col].mode()[0], inplace=True)
```

	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	\
0	2	13.4	22.9	0.6	4.8	8.4	13	
1	2	7.4	25.1	0.0	4.8	8.4	14	
2	2	12.9	25.7	0.0	4.8	8.4	15	
3	2	9.2	28.0	0.0	4.8	8.4	4	
4	2	17.5	32.3	1.0	4.8	8.4	13	

	WindGustSpeed	WindDir9am	WindDir3pm	...	Humidity9am	Humidity3pm	\
0	44.0	13	14	...	71.0	22.0	
1	44.0	6	15	...	44.0	25.0	
2	46.0	13	15	...	38.0	30.0	
3	24.0	9	0	...	45.0	16.0	
4	41.0	1	7	...	82.0	33.0	

	Pressure9am	Pressure3pm	Cloud9am	Cloud3pm	Temp9am	Temp3pm	RainToday	\
0	1007.7	1007.1	8.0	5.0	16.9	21.8	0	
1	1010.6	1007.8	5.0	5.0	17.2	24.3	0	
2	1007.6	1008.7	5.0	2.0	21.0	23.2	0	
3	1017.6	1012.8	5.0	5.0	18.1	26.5	0	
4	1010.8	1006.0	7.0	8.0	17.8	29.7	0	

	RainTomorrow
0	0
1	0
2	0
3	0
4	0

[5 rows x 22 columns]

Q2. Train-Test Split

```
#Q2

from sklearn.model_selection import train_test_split

X = df.drop("RainTomorrow", axis=1)
y = df["RainTomorrow"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
... Training Data: (116368, 21)
... Testing Data: (29092, 21)
```

Q3. Building Logistic Regression Model

```
#Q3
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop("RainTomorrow", axis=1)
y = df["RainTomorrow"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale the features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=5000)

model.fit(X_train, y_train)

print("Model Trained Successfully")
```

... Model Trained Successfully

Q4. Making Predictions

▶ #Q4

```
y_pred = model.predict(X_test)

print("Actual Values")
print(y_test.head())

print("Predicted Values")
print(y_pred[:5])
```

```
*** Actual Values
100721    1
30234     0
68427     0
28624     0
31173     0
Name: RainTomorrow, dtype: int64
Predicted Values
[1 0 0 0 0]
```

Q5. Confusion Matrix

▶ #Q5

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix")

print(cm)
```

```
*** Confusion Matrix
[[21488  1184]
 [ 3379 3041]]
```

Q6. Model Evaluation Metrics



#Q6

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report

print("Accuracy :", accuracy_score(y_test, y_pred))

print("Precision :", precision_score(y_test, y_pred))

print("Recall :", recall_score(y_test, y_pred))

print("F1 Score :", f1_score(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

```
*** Accuracy : 0.843152756771621
    Precision : 0.7197633136094674
    Recall : 0.47367601246105917
    F1 Score : 0.5713480507280413
```

	precision	recall	f1-score	support
0	0.86	0.95	0.90	22672
1	0.72	0.47	0.57	6420
accuracy			0.84	29092
macro avg	0.79	0.71	0.74	29092
weighted avg	0.83	0.84	0.83	29092

Q7. Saving the Model

```
#Q7

import joblib

joblib.dump(model, "weather_model.pkl")

print("Model Saved Successfully")
```

```
... Model Saved Successfully
```

Q8. Loading & Testing Saved Model

```
#Q8

import joblib

# Load the saved model
loaded_model = joblib.load("weather_model.pkl")

# Predict first 5 test samples
prediction = loaded_model.predict(X_test[:5])

print("Predictions:")
print(prediction)
```

```
... Predictions:
[1 0 0 0 0]
```

Q9. Streamlit Input Interface

```
import streamlit as st
```

```
st.title("Weather Prediction")
```

```
MinTemp = st.number_input("Minimum Temperature")
```

```
MaxTemp = st.number_input("Maximum Temperature")
```

```
Rainfall = st.number_input("Rainfall")
```

```
Evaporation = st.number_input("Evaporation")
```

```
Sunshine = st.number_input("Sunshine")
```

```
WindGustSpeed = st.number_input("Wind Gust Speed")
```

```
Humidity9am = st.number_input("Humidity 9AM")
```

```
Humidity3pm = st.number_input("Humidity 3PM")
```

```
Pressure9am = st.number_input("Pressure 9AM")
```

```
Pressure3pm = st.number_input("Pressure 3PM")
```

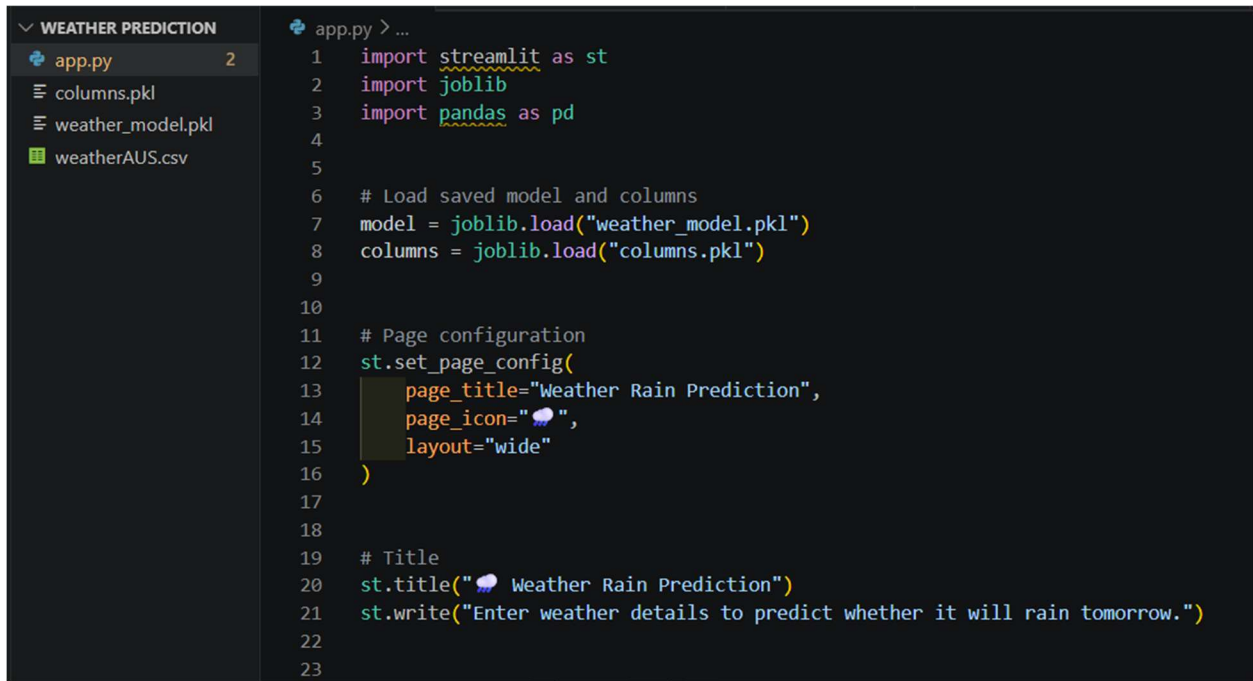
```
Temp9am = st.number_input("Temperature 9AM")
```

```
Temp3pm = st.number_input("Temperature 3PM")
```

```
if st.button("Predict"):
```

```
    st.write("Predict button clicked!")
```


Q10. Complete Streamlit Web App



```

WEATHER PREDICTION
app.py 2
columns.pkl
weather_model.pkl
weatherAUS.csv

app.py > ...
1  import streamlit as st
2  import joblib
3  import pandas as pd
4
5
6  # Load saved model and columns
7  model = joblib.load("weather_model.pkl")
8  columns = joblib.load("columns.pkl")
9
10
11 # Page configuration
12 st.set_page_config(
13     page_title="Weather Rain Prediction",
14     page_icon="🌧️",
15     layout="wide"
16 )
17
18
19 # Title
20 st.title("🌧️ Weather Rain Prediction")
21 st.write("Enter weather details to predict whether it will rain tomorrow.")
22
23

```

```

24 # Create input fields
25 input_data = {}
26
27 col1, col2, col3 = st.columns(3)
28
29 for i, feature in enumerate(columns):
30
31     if i % 3 == 0:
32         with col1:
33             input_data[feature] = st.number_input(
34                 feature,
35                 value=0.0
36             )
37
38     elif i % 3 == 1:
39         with col2:
40             input_data[feature] = st.number_input(
41                 feature,
42                 value=0.0
43             )
44
45     else:
46         with col3:
47             input_data[feature] = st.number_input(
48                 feature,
49                 value=0.0
50             )
51
52
53 # Prediction button
54 if st.button("🔍 Predict Weather"):
55
56     # Convert input into dataframe
57     input_df = pd.DataFrame([input_data])
58

```

```
58
59 # Prediction
60 prediction = model.predict(input_df)
61
62
63 if prediction[0] == 1:
64     st.error("☁ Prediction: Rain Tomorrow")
65 else:
66     st.success("☀ Prediction: No Rain Tomorrow")
67
68
69 # Footer
70 st.write("---")
71 st.caption("Weather Prediction using Logistic Regression Machine Learning Model")
```

OUTPUTS

1. App.py output

```
C:\Users\tanve\Desktop\weather_prediction>python -m streamlit run app.py
```

You can now view your Streamlit app in your browser.

Local URL: <http://localhost:8502>

Network URL: <http://192.168.1.11:8502>

2. Streamlit app home page

Deploy

Enter weather details to predict whether it will rain tomorrow.

Location	MinTemp	MaxTemp
0.00 - +	0.00 - +	0.00 - +
Rainfall	Evaporation	Sunshine
0.01 - +	0.04 - +	0.02 - +
WindGustDir	WindGustSpeed	WindDir9am
0.01 - +	0.01 - +	0.02 - +
WindDir3pm	WindSpeed9am	WindSpeed3pm
0.01 - +	0.02 - +	0.03 - +
Humidity9am	Humidity3pm	Pressure9am
0.01 - +	0.04 - +	0.02 - +
Pressure3pm	Cloud9am	Cloud3pm
0.00 - +	0.10 - +	0.02 - +
Temp9am	Temp3pm	RainToday
0.27 - +	0.02 - +	0.04 - +

 Predict Weather

 Prediction: No Rain Tomorrow

Weather Prediction using Logistic Regression Machine Learning Model



Enter weather details to predict whether it will rain tomorrow.

Location	MinTemp	MaxTemp
0.00 - +	0.00 - +	0.25 - +
Rainfall	Evaporation	Sunshine
0.01 - +	0.04 - +	0.02 - +
WindGustDir	WindGustSpeed	WindDir9am
0.01 - +	0.01 - +	0.02 - +
WindDir3pm	WindSpeed9am	WindSpeed3pm
0.01 - +	0.02 - +	0.03 - +
Humidity9am	Humidity3pm	Pressure9am
0.01 - +	0.04 - +	0.02 - +
Pressure3pm	Cloud9am	Cloud3pm
0.00 - +	0.10 - +	0.02 - +
Temp9am	Temp3pm	RainToday
17.00 - +	23.00 - +	0.04 - +

 Predict Weather

 Prediction: Rain Tomorrow

Weather Prediction using Logistic Regression Machine Learning Model

Enter weather details to predict whether it will rain tomorrow.

Location	MinTemp	MaxTemp
0.07 - +	0.00 - +	0.21 - +
Rainfall	Evaporation	Sunshine
0.05 - +	0.04 - +	0.02 - +
WindGustDir	WindGustSpeed	WindDir9am
0.01 - +	0.01 - +	0.02 - +
WindDir3pm	WindSpeed9am	WindSpeed3pm
0.08 - +	0.02 - +	0.03 - +
Humidity9am	Humidity3pm	Pressure9am
0.01 - +	0.03 - +	0.02 - +
Pressure3pm	Cloud9am	Cloud3pm
0.00 - +	0.10 - +	0.02 - +
Temp9am	Temp3pm	RainToday
30.00 - +	28.00 - +	0.15 - +

 Predict Weather

 Prediction: Rain Tomorrow

Weather Prediction using Logistic Regression Machine Learning Model

SHR

➤ Conclusion:

- The Weather Prediction model was successfully developed using a Machine Learning Logistic Regression algorithm.
- The weather dataset was cleaned, processed, and used to train the model.
- The trained model was saved as a .pkl file and integrated with a Streamlit web application.
- The application takes weather parameters as input and predicts whether it will rain tomorrow or not.
- Feature matching was maintained by using the saved columns.pkl file, which ensures the model receives the correct 21 input features.
- The project demonstrates how Machine Learning models can be deployed into a user-friendly application for real-time predictions.
- This system can help users understand weather patterns and make better decisions based on predicted rainfall conditions.